



Урок 5 > Выбор подходящих баз данных

> Глоссарий

Требования ACID — это набор требований, которые обеспечивают сохранность ваших данных.

- **Atomicity** — Атомарность. Гарантирует, что каждая транзакция будет выполнена полностью или не будет выполнена совсем. Не допускаются промежуточные состояния.
- **Consistency** — Согласованность. Это свойство вытекает из предыдущего. Благодаря тому, что транзакция не допускает промежуточных результатов, база остается консистентной. Есть такое определение транзакции: «Упорядоченное множество операций, переводящих базу данных из одного согласованного состояния в другое». То есть до выполнения операции и после база остается консистентной.
- **Isolation** — Изолированность. Во время выполнения транзакции параллельные транзакции не должны оказывать влияния на её результат.
- **Durability** — Надёжность. Если пользователь получил подтверждение от системы, что транзакция выполнена, он может быть уверен, что сделанные им изменения не будут отменены из-за какого-либо сбоя. Обесточилась

система, произошел сбой в оборудовании? На выполненную транзакцию это не повлияет.

> Основные виды БД

- **Реляционные** — базы данных, записи в которых хранятся в виде набора таблиц и заранее определённых связей между ними. К таким БД относятся все СУБД, а также базы удовлетворяющие ACID-требованиям.
 - **БД типа key-value** — поддерживают и хранят только наиболее простые операции с записями по ключу.
 - **Колоночные БД** — данные хранятся в виде набора колонок, не связанных между собой. Каждая запись, которую нужно сохранить в базу, разбивается на части, каждая часть записывается в свою колонку.
 - **Документные БД** — хранят данные в виде сложных древовидных структур типа JSON.
-

Выделяют ещё и другие типы БД, но они используются крайне редко и для очень специфичных задач:

- [Временные \(темпоральные\) БД](#)
- [Графовые БД](#)
- и другие

> Реляционные базы данных

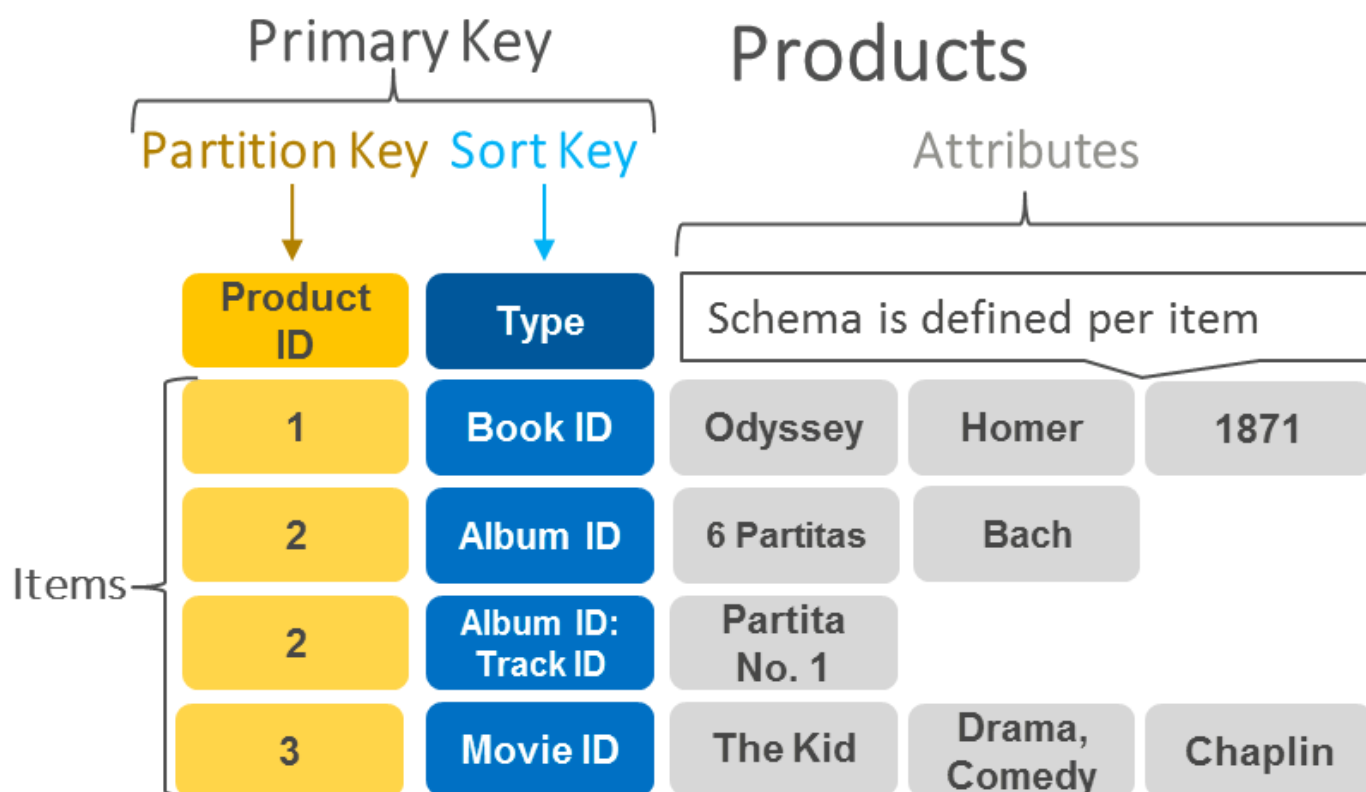
Реляционные БД – базы данных, основанные на реляционной модели, подразумевающей организацию данных в виде таблиц (строк-записей и столбцов-атрибутов), каждая запись в которой имеет уникальный ключ и может в атрибутах ссылаться на ключи в других таблицах. Можно считать это «базой данных по умолчанию», т.к. при создании MVP вашего проекта вы наверняка подключите БД вроде SQLite, Postgres, MySQL и т.п. для хранения ваших записей.

Все реляционные базы данных соблюдают ACID-требования.

> БД вида key-value

База данных на основе пар «ключ-значение» — это тип нереляционных баз данных, в котором для хранения данных используется простой метод «ключ-значение». База данных на основе пар «ключ-значение» хранит данные как совокупность пар «ключ-значение», в которых ключ служит уникальным идентификатором. Как ключи, так и значения могут представлять собой что угодно: от простых до сложных составных объектов.

Базы данных с использованием пар «ключ-значение» поддерживают высокую разделяемость и обеспечивают беспрецедентное горизонтальное масштабирование, недостижимое при использовании других типов баз данных. Например, Amazon DynamoDB выделяет дополнительные разделы на таблицу, если существующий раздел заполняется до предела и требуется больше пространства для хранения.



Пример данных, хранящихся в виде пар «ключ-значение» в DynamoDB

Примеры использования

Хранилище сессий

Основанное на сессиях приложение (например, интернет-приложение) запускает сессию, когда пользователь входит в систему. Оно активно до тех пор, пока пользователь не выйдет из системы или не истечет время сессии. В течение этого периода приложение хранит все связанные с сессией данные либо в основной памяти, либо в базе данных. Данные сессии могут включать информацию профиля пользователя, сообщения, индивидуальные данные и темы, рекомендации, таргетированные рекламные кампании и скидки. Каждая сессия пользователя имеет уникальный идентификатор. Данные сессий всегда запрашиваются только по первичному ключу, поэтому для их хранения отлично подходит быстрое хранилище пар «ключ-значение». В целом базы данных на основе пар «ключ-значение» могут снижать накладные расходы в расчёте на страницу по сравнению с реляционными базами данных.

Корзина интернет-магазина

Во время праздничного сезона покупок сайт интернет-магазина может получать миллиарды заказов за считанные секунды. Используя базы данных на основе пар «ключ-значение», можно обеспечить необходимое масштабирование при существенном увеличении объемов данных и чрезвычайно интенсивных изменениях состояния. Такие базы данных позволяют одновременно обслуживать миллионы пользователей благодаря распределённой обработке и хранению данных. Базы данных на основе пар «ключ-значение» также обладают встроенной избыточностью, что позволяет справляться с потерей узлов хранилища.

Из примеров:

- [Redis](#)
- [Memcache](#)

> Колоночные базы данных

Колоночные БД призваны решить проблему неэффективной работы традиционных СУБД в аналитических системах и системах с подавляющим большинством операций типа «чтение». Они позволяют на более дешёвом и маломощном оборудовании получить прирост скорости выполнения запросов в 5, 10 и иногда даже в 100 раз, при этом, благодаря компрессии, данные будут занимать на диске в 5-10 раз меньше, чем в случае с традиционными БД.

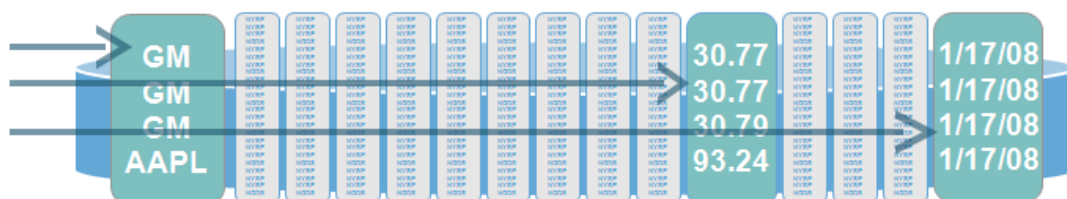
У колоночных БД есть и недостатки — они медленно работают на запись, не подходят для транзакционных систем и, как правило, ввиду «молодости» имеют ряд ограничений для разработчика, привыкшего к развитым традиционным БД.

Обычно колоночные БД применяются в аналитических системах класса business intelligence (ROLAP) и аналитических хранилищах данных (data warehouses).

Причём объёмы данных могут быть достаточно большими — есть примеры по 300-500ТБ и даже случаи с >1ПБ данных.

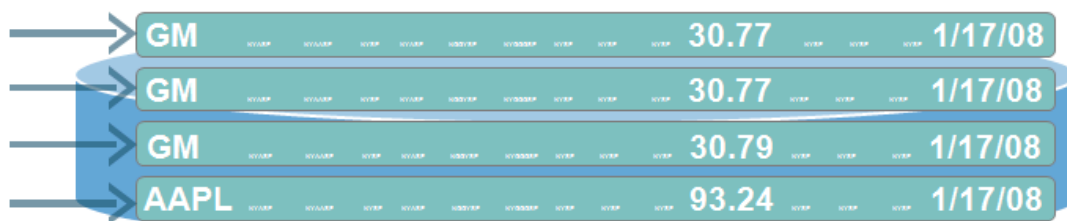
Хранение по столбцам

Чтение 3х столбцов



Хранение по строкам

Чтение всех столбцов



Примеры БД:

- [Hbase](#)
- [Vertica](#)
- [ClickHouse](#)

Дополнительные ссылки:

[Перевод статьи М. Стоунбрекера "«One Size Fits All»: An Idea Whose Time Has Come and Gone"](#)

> Документные базы данных

Документная база данных — это тип нереляционных баз данных, предназначенный для хранения и запроса данных в виде документов в формате, подобном JSON.

Документные базы данных позволяют разработчикам хранить и запрашивать данные в БД с помощью той же документной модели, которую они используют в коде приложения. Гибкий, полуструктурированный, иерархический характер

документов и документных баз данных позволяет им развиваться в соответствии с потребностями приложений.

Документная модель хорошо работает в таких примерах использования, как каталоги, пользовательские профили и системы управления контентом, где каждый документ уникален и изменяется со временем. Документные базы данных обеспечивают гибкость индексации, производительность выполнения стандартных запросов и аналитику наборов документов.

В следующем примере документ в формате, подобном JSON, описывает книгу.

```
1 [
2   {
3     "year" : 2013,
4     "title" : "Turn It Down, Or Else!",
5     "info" : {
6       "directors" : [ "Alice Smith", "Bob Jones"],
7       "release_date" : "2013-01-18T00:00:00Z",
8       "rating" : 6.2,
9       "genres" : ["Comedy", "Drama"],
10      "image_url" : "http://ia.media-
imdb.com/images/N/09ERWAU7FS797AJ7LU8HN09AMUP908RL1o5JF90EWR7LJKQ7@@._V1_SX400_.jpg",
11      "plot" : "A rock band plays their music at high volumes, annoying the
neighbors.",
12      "actors" : ["David Matthewman", "Jonathan G. Neff"]
13    }
14  },
15  {
16    "year": 2015,
17    "title": "The Big New Movie",
18    "info": {
19      "plot": "Nothing happens at all.",
20      "rating": 0
21    }
22  }
23 ]
```

Примеры использования

Управление контентом

Документная база данных — отличный выбор для приложений управления контентом, таких как платформы для блогов и размещения видео. При

использовании документной базы данных каждая сущность, отслеживаемая приложением, может храниться как отдельный документ.

Такой вид базы данных позволяет разработчику с удобством обновлять приложение при изменении требований. Кроме того, если необходимо изменить модель данных, то требуется обновление только затронутых этим изменением документов. Для внесения изменений нет необходимости обновлять схему и прерывать работу базы данных.

Каталоги

Документные базы данных эффективны для хранения каталожной информации. Например, в приложениях для интернет-коммерции разные товары обычно имеют различное количество атрибутов. Управление тысячами атрибутов в реляционных базах данных неэффективно. Кроме того, количество атрибутов влияет на производительность чтения. При использовании документной базы данных атрибуты каждого товара можно описать в одном документе, что упрощает управление и повышает скорость чтения. Изменение атрибутов одного товара не повлияет на другие товары.

Из примеров:

- [MongoDB](#)
- [CouchDB](#)
- [Amazon DocumentDB](#)

> CAP- и PACELC-теоремы

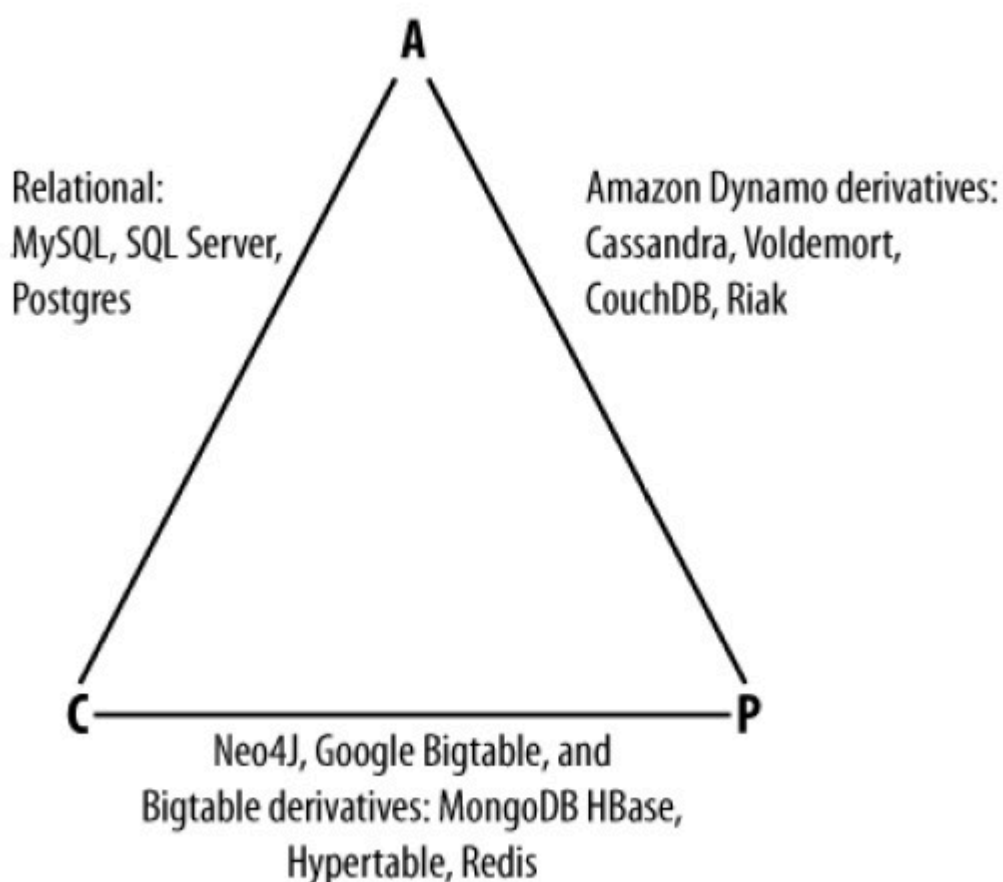
CAP теорема

Эта теорема была представлена на симпозиуме по принципам распределённых вычислений в 2000 году Эриком Брюером. В 2002 году Сет Гилберт и Нэнси Линч из MIT опубликовали формальное доказательство гипотезы Брюера, сделав её теоремой.

В чем заключается теорема

В CAP говорится, что в распределённой системе возможно выбрать только два из трёх свойств:

- C (consistency) — согласованность. Каждое чтение даст вам самую последнюю запись.
- A (availability) — доступность. Каждый узел (не упавший) всегда успешно выполняет запросы (на чтение и запись).
- P (partition tolerance) — устойчивость к распределению. Даже если между узлами нет связи, они продолжают работать независимо друг от друга.



PACELC-теорема

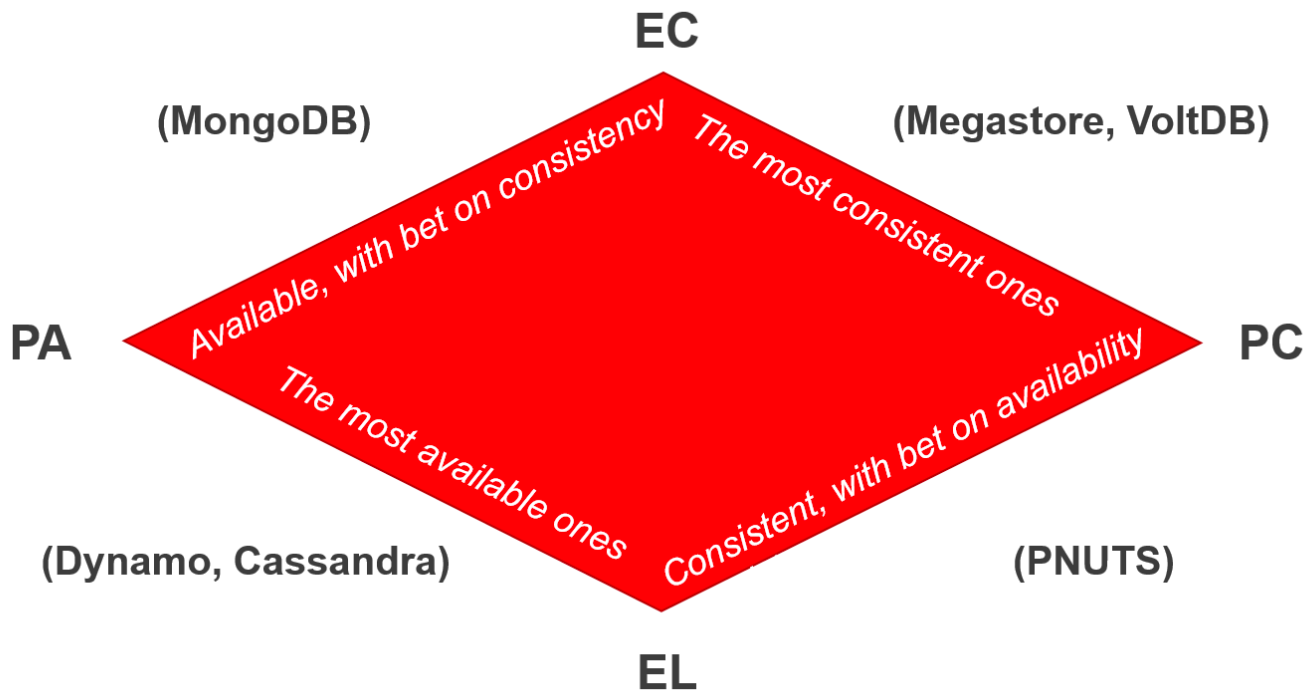
Теорема PACELC была впервые описана и формализована Даниелом Дж. Абади из Йельского университета в 2012 году. Поскольку теорема PACELC основана на CAP, она также использует его определения.

Вся теорема сводится к `if P -> (A or C), else (L or C)`, или же:

- В случае наличия разделения (P) надо выбирать между доступностью (A) и согласованностью (C).

- Иначе (Else) мы можем дополнительно выбирать между меньшей задержкой (L) и согласованностью (C).

Latency — это время, за которое клиент получит ответ и которое регулируется каким-либо уровнем consistency. Latency в некотором смысле представляет собой степень доступности.



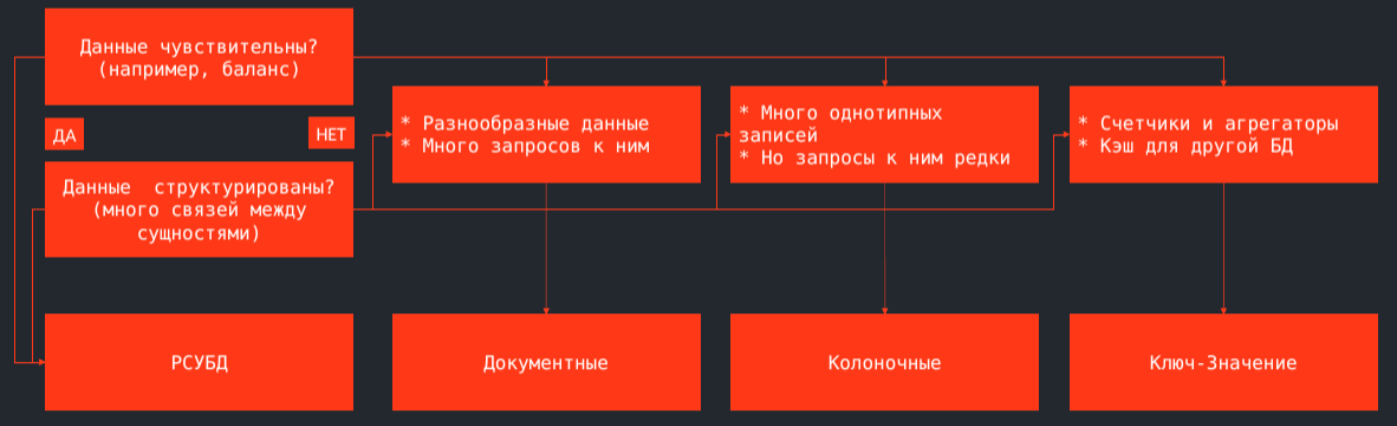
> Выбор подходящей БД

При проектировании подсистемы требуется определиться, что из C, A и P нам важнее:

- У согласованных и доступных систем (CA) есть сложности с устойчивостью к разделению, которые частично решаются созданием реплик, а также такие системы масштабируют вертикально (мощнее железо), чтобы изначально избежать разделения. Именно сюда попадают RDBMS с ACID, которые всегда кстати для хранения чувствительных данных.
- В согласованных и устойчивых к разделению системах (CP) могут быть проблемы с доступностью, но зато данные в разделённых нодах всегда корректны и выглядят одинаково. Примеры таких БД есть в разных семействах — колоночная HBase, документная Mongo и Redis.
- В доступных и устойчивых к разделению системах (AP) нет строгой согласованности, но она так или иначе достигается, чего бывает достаточно

для крупных систем (например, чей-то новый пост на другом конце света увидят на минуту позже). Примеры опять же есть разные — колоночная Cassandra, документная CouchDB и Dynamo. В противовес к строгим ACID выделяют BASE (basic availability, soft state, eventual consistency).

Как же выбрать подходящую БД для нашей подсистемы?



> Сокращатель ссылок

Какие данные храним:

- для каждой короткой ссылки храним созданный ключ, даты создания/протухания, опционально id пользователя;
- можем хранить профиль пользователя с id и типичными анкетными данными (имя, почта, дата создания, время последнего посещения).

Вывод:

- есть всего две легковесные, просто связанные таблицы, чувствительных данных нет (денежные транзакции), а записей поступает миллиарды;
- ACID не критичен, JOIN-ы можно сделать программно, следовательно, колоночная БД типа Cassandra будет в самый раз.

> Хостинг текстов

Какие данные храним:

- для каждого текста храним созданный id и сам контент, даты создания/протухания, опционально id пользователя;

- можем хранить профили пользователя с id и типичными анкетными данными (имя, почта, даты создания и последнего посещения);
- если тексты достаточно большие, с опциональной привязкой картинок, можно подумать над их хранением отдельно.

Вывод:

- есть всего две легковесные, просто связанные таблицы, чувствительных данных нет (денежные транзакции), а записей поступает миллиарды;
- ACID не критичен, JOIN-ы можно сделать программно, следовательно, колоночная БД типа Cassandra будет в самый раз. Большие тексты и другие медиафайлы можно хранить в отдельном хранилище типа Amazon S3, повышая лимиты при необходимости.

> Облачный диск

Какие данные храним:

- метаданные: информация про пользователей, устройства, которыми они пользуются, последнее время посещения, и файлы, разбитые на блоки вместе с хешами, для дедупликации;
- сами блоки файлов как большие бинарные объекты в отдельном файловом хранилище.

Вывод:

- метаданные про пользователей и используемые файлы будет легче хранить в РСУБД с ACID из-за чувствительности (оплата подписки за сервис);
- с другой стороны, из-за большого количества блоков и скорости их прирастания, будет сложнее масштабироваться;
- можно рассмотреть колоночные БД с программной «ACID-изацией» критических участков системы, связанных с синхронизацией и т.д.;
- файловые блоки можем хранить как объекты в отдельном хранилище типа AmazonS3, повышая лимиты при необходимости.

> Фотоаппы

Какие данные храним:

- стандартный профиль пользователя (имя, почта, аватарка, даты создания и последнего посещения);
- фотографии (медиафайлы) и метаданные для них (id автора, теги, геометка, дата публикации и т.п.);
- таблицы подписок на посты пользователей.

Вывод:

- т.к. для создания ленты пользователей (и рекомендаций) нужны JOIN-ы, очевидным выбором будет РСУБД типа Postgres, MySQL и т.д.;
- но популярные сервисы с фотографиями так или иначе придётся масштабировать, и с РСУБД это делать сложнее;
- можно рассмотреть сценарий с «программными» JOIN-ами, когда все метаданные постов хранятся в колоночной БД типа Cassandra;
- ленту пользователя с рекомендациями и постами друзей можно также предсчитать программно, и кешировать в key-value хранилищах типа Redis.

> Телеграм

Какие данные храним:

- онлайн данные об установленных соединениях, открытых чатах, каналах и т.п.;
- данные о пользователе (профиль, аватарки, время посещения, настройки приложения и т.п.);
- история чатов, изображения и прочие прикрепленные медиа и просто файлы.

Вывод:

- для онлайн данных нужно по ключам user id получать и изменять сведения о соединениях, для чего подойдёт key-value БД типа Redis;
- данные о пользователях и их связях (скорее всего «контактах») можно хранить в реляционных БД вроде Postgres, а аватарки в S3-подобных хранилищах;

- т.к. количество сообщений будет большим, всю историю сообщений можно хранить в колоночных БД, а недавние сообщения выкладывать заранее в Redis.

> Твиттер

Какие данные храним:

- стандартный профиль пользователя (имя, почта, аватарка, дата создания и последнего посещения);
- текст твита, медиафайлы + метаданные (id автора, теги, геометка, даты, лайки и т.п.);
- таблицы подписок и избранных постов.

Вывод:

- в целом, требования очень похожи на фотоаппы – профили, подписки и лайки можно хранить в Postgres, а твиты в HBase + Redis
- ленту пользователя можно также предпосчитать и кешировать опять же в key-value хранилище типа Redis

> Нетфликс

Какие данные храним:

- стандартные профили пользователей, справочную информацию о фильмах, оценки и добавление в избранное;
- разнообразная статистика по просмотрам для аналитики и построения рекомендаций;
- сами фильмы в разных разрешениях, звуковые дорожки и субтитры к ним.

Вывод:

- для хранения стандартных табличных данных можно использовать РСУБД типа Postgres, частые фильмы положить в кеш (Redis);
- для аналитики и рекомендаций нужно будет собирать много логов в колоночную БД типа Cassandra;

- готовые рекомендации — в Redis. Сами видеофайлы и сопроводительные материалы нужно будет хранить в хранилищах типа S3, желательно геораспределённых.

> Сервис такси

Какие данные храним:

- профили пользователей, настройки приложений;
- онлайн данные о позициях пользователей и текущих поездках;
- офлайн данные об истории поездок, чат с поддержкой.

Вывод:

- стандартные табличные данные с профилями и настройками — подойдёт РСУБД типа Postgres;
- данные о подключённых пользователях и текущих поездках храним в key-value БД, позиции водителей в специализированной гео-БД;
- при окончании поездки нуждающиеся в ACID обновления делаем в Postgres и кладём поездку в Cassandra, а недавние храним в Redis.